

Model Diagnostics and Visualization

GAM.jl Contributors

Table of contents

Introduction	1
Setup	2
Simulate Gu–Wahba example data	2
Fit the GAM	3
Model overview	3
Smooth estimates	4
Partial residuals	6
Derivatives	7
Model diagnostics with appraise	10
Posterior simulation	12
Smooth samples	12
Fitted samples	14
Concurvity	14
Basis dimension check	15
Summary	15

Introduction

This vignette demonstrates **gratia**-style model diagnostics and visualization tools available in GAM.jl. These functions mirror the R [gratia](#) package, providing tidy interfaces for inspecting fitted GAMs.

We fit a GAM to the classic Gu & Wahba (1991) simulated example with four smooth terms of varying complexity, then explore the full diagnostic toolkit.

Setup

```
using GAM
using StatsAPI: fitted, predict, residuals
using DataFrames
using CSV
using Random
using Statistics
using Plots
```

Simulate Gu–Wahba example data

The test functions are:

- $f_0(x) = 2 \sin(\pi x)$
- $f_1(x) = \exp(2x)$
- $f_2(x) = 0.2x^{11}(10(1-x))^6 + 10(10x)^3(1-x)^{10}$
- $f_3(x) = 0$ (null smooth)

```
f0(x) = 2 * sin( * x)
f1(x) = exp(2 * x)
f2(x) = 0.2 * x^11 * (10 * (1 - x))^6 + 10 * (10 * x)^3 * (1 - x)^10
f3(x) = 0.0

df = CSV.read("data.csv", DataFrame)
n = nrow(df)
x0 = df.x0; x1 = df.x1; x2 = df.x2; x3 = df.x3
y = df.y
first(df, 5)
```

	y	x0	x1	x2	x3
	Float64	Float64	Float64	Float64	Float64
1	2.99265	0.914806	0.0227	0.909048	0.40188
2	4.69717	0.937075	0.51324	0.899925	0.432214
3	13.9358	0.28614	0.630726	0.192349	0.663604
4	5.71329	0.830448	0.418772	0.53229	0.182369
5	7.63407	0.641746	0.879266	0.522125	0.838339

Fit the GAM

```
m = gam(@gam_formula(y ~ s(x0, k=10, bs=:cr) + s(x1, k=10, bs=:cr) +  
                      s(x2, k=10, bs=:cr) + s(x3, k=10, bs=:cr)), df)
```

Generalized Additive Model

Formula: y ~ 1

Family: Normal

Link: IdentityLink

Method: REML

Parametric coefficients:

	Coef.	Std. Error	t	Pr(> t)
(Intercept)	7.49514	0.105069	71.34	<1e-99

Approximate significance of smooth terms:

Smooth	edf	Ref.df
s(x0,bs=cr)	3.43	9
s(x1,bs=cr)	3.20	9
s(x2,bs=cr)	7.83	9
s(x3,bs=cr)	1.89	9

R² (adj) = 0.685 Deviance explained = 69.8%
Scale est. = 4.4158 n = 400

Model overview

The `overview` function provides a tidy summary of all smooth terms, including basis type, dimension, basis size, effective degrees of freedom (EDF), and EDF-to-k ratio.

```
ov = overview(m)
```

GAM Overview: 4 smooth term(s)

Smooth	Type	Dim	k	EDF	k-ratio
s(x0,bs=cr)	CubicSpline	1	9	3.43	0.381
s(x1,bs=cr)	CubicSpline	1	9	3.20	0.355
s(x2,bs=cr)	CubicSpline	1	9	7.83	0.870
s(x3,bs=cr)	CubicSpline	1	9	1.89	0.210

Smooth estimates

`smooth_estimates` evaluates each smooth function on a regular grid with pointwise standard errors. This is the foundation for plotting smooth effects.

```
se = smooth_estimates(m; n=200)
```

SmoothEstimates: 4 smooth(s), 800 evaluation points

```
s(x0,bs=cr): 200 points
s(x1,bs=cr): 200 points
s(x2,bs=cr): 200 points
s(x3,bs=cr): 200 points
```

We plot each estimated smooth with ± 2 SE confidence bands, overlaying the true function (centered to have zero mean, for identifiability):

```
true_fns = [f0, f1, f2, f3]
x_vars = [:x0, :x1, :x2, :x3]
x_data = [x0, x1, x2, x3]
labels = ["f (x ) = 2sin( x)", "f (x ) = exp(2x)", "f (x ) = Gu-Wahba", "f (x ) = 0"]

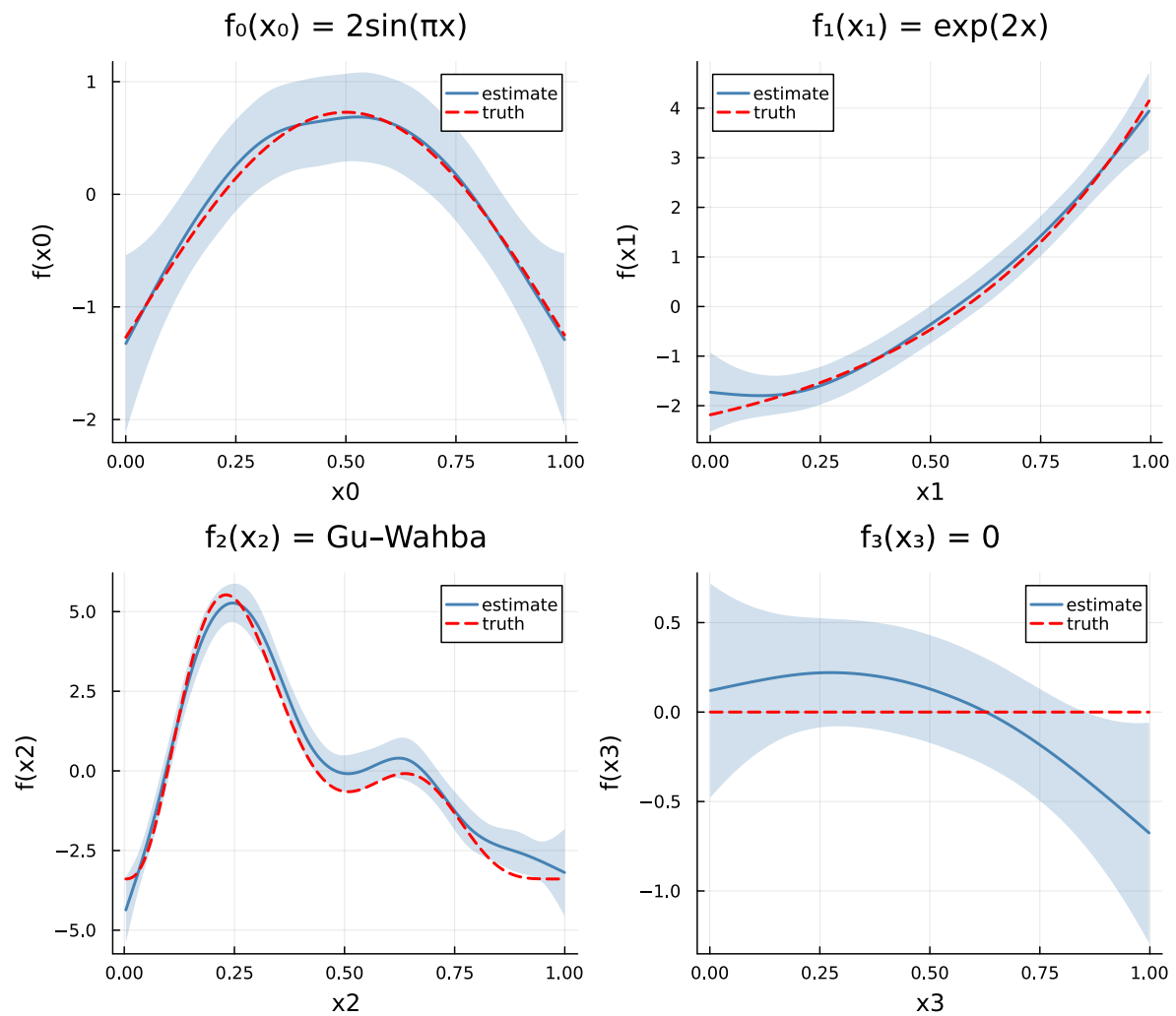
plots_se = []
for (i, xvar) in enumerate(x_vars)
    se_i = smooth_estimates(m; select=string(xvar), n=200)
    xgrid = se_i.covariates[xvar]
    est = se_i.estimate
    se_vals = se_i.se

    # True function on grid, centered
    f_grid = true_fns[i].(xgrid)
    f_grid = f_grid .- mean(f_grid)
```

```

p = plot(xgrid, est;
        ribbon=2 .* se_vals, fillalpha=0.25, color=:steelblue,
        linewidth=2, label="estimate",
        xlabel=string(xvar), ylabel="f($(xvar))",
        title=labels[i])
plot!(p, xgrid, f_grid;
      linestyle=:dash, linewidth=2, color=:red, label="truth")
push!(plots_se, p)
end
plot(plots_se...; layout=(2, 2), size=(800, 700))

```



Select a specific smooth:

```
se_x0 = smooth_estimates(m; select="s(x0)")
println("Grid points: $(length(se_x0.estimate)), SE range: [$(round(minimum(se_x0.se), digits=3),
```

Grid points: 100, SE range: [0.195, 0.392]

Partial residuals

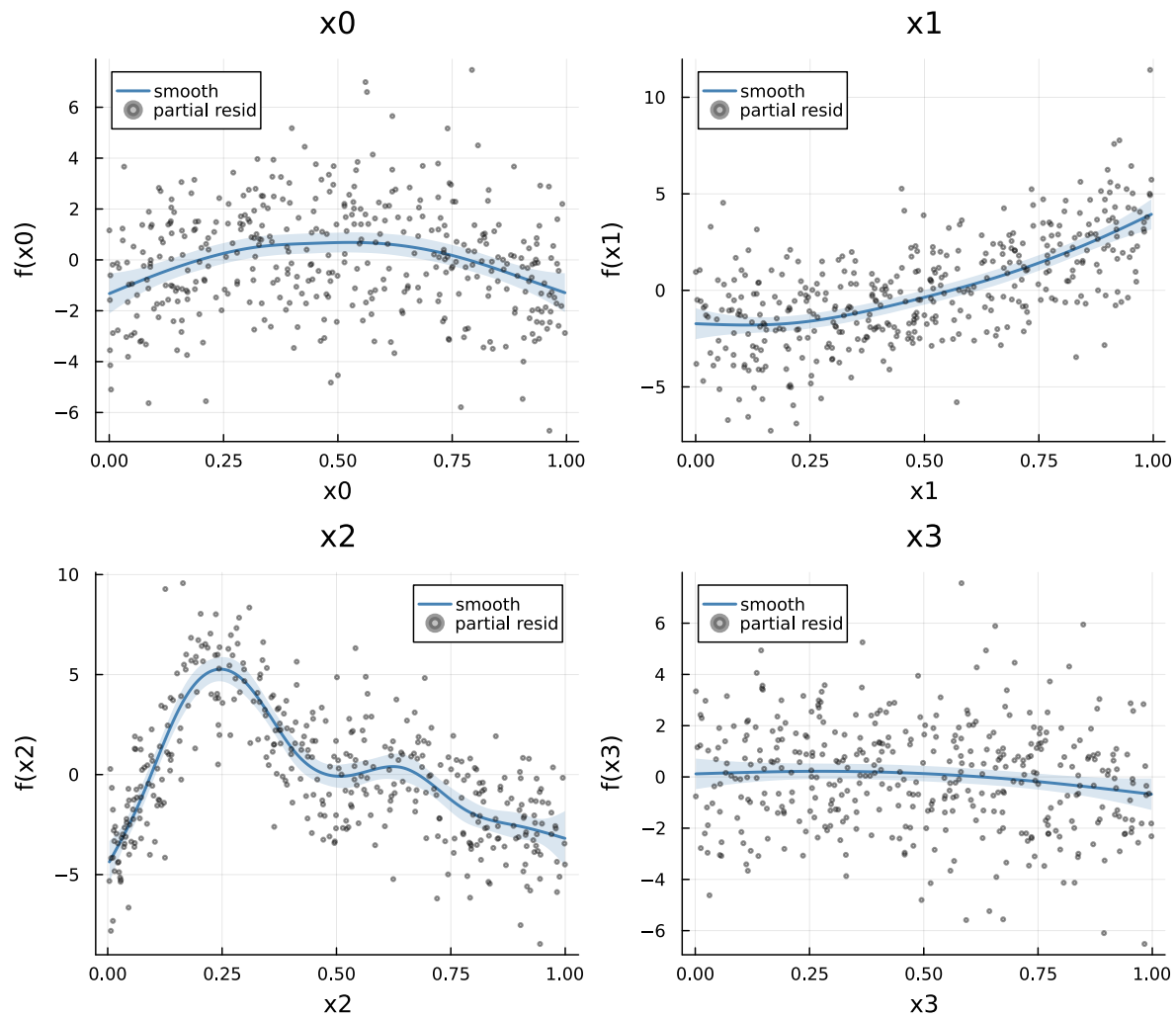
`partial_residuals` computes $\hat{f}_j(x_{ij}) + \hat{\varepsilon}_i$ for each smooth, useful for assessing fit quality by overlaying partial residuals on smooth estimates.

```
pr = partial_residuals(m)

plots_pr = []
for (i, xvar) in enumerate(x_vars)
    se_i = smooth_estimates(m; select=string(xvar), n=200)
    xgrid = se_i.covariates[xvar]
    est = se_i.estimate
    se_vals = se_i.se

    label_key = m.smooths[i].spec.label
    xvals, resids = pr[label_key]

    p = plot(xgrid, est;
        ribbon=2 .* se_vals, fillalpha=0.2, color=:steelblue,
        linewidth=2, label="smooth",
        xlabel=string(xvar), ylabel="f($(xvar))",
        title=string(xvar))
    scatter!(p, xvals, resids;
        markersize=1.5, alpha=0.4, color=:grey40, label="partial resid")
    push!(plots_pr, p)
end
plot(plots_pr...; layout=(2, 2), size=(800, 700))
```



Derivatives

`derivatives` computes finite-difference derivatives of smooth terms with confidence intervals, useful for identifying regions of significant change.

```
d = derivatives(m; order=1, type=:central, n=200, level=0.95)
```

```
DerivativeEstimates (order=1, type=central): 4 smooth(s)
```

We highlight regions where the confidence interval excludes zero (i.e., statistically significant change):

```

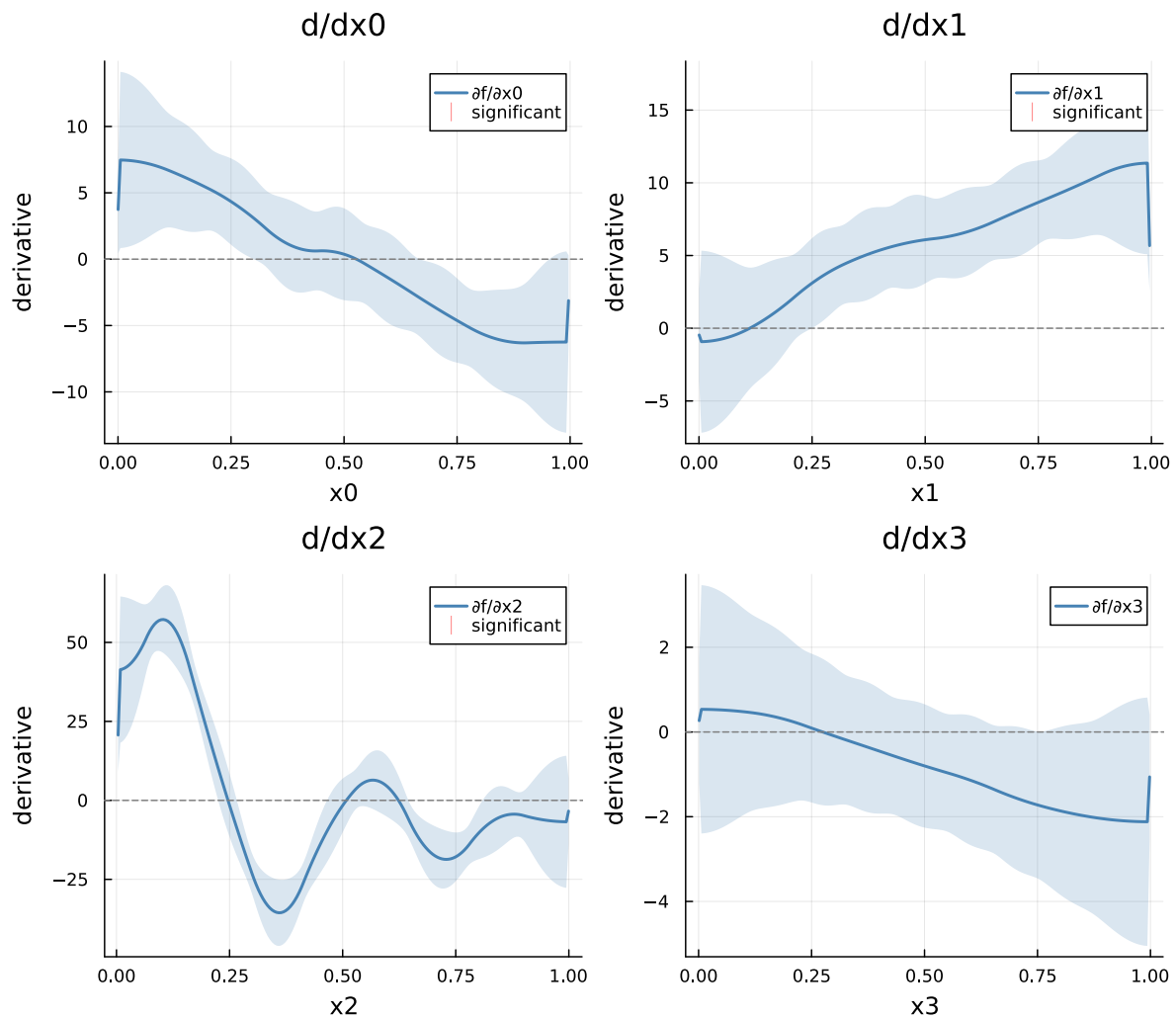
plots_d = []
for (i, xvar) in enumerate(x_vars)
    d_i = derivatives(m; select=string(xvar), order=1, type=:central, n=200)
    xgrid = d_i.x
    deriv = d_i.derivative
    lower = d_i.lower
    upper = d_i.upper

    # Color significant regions
    sig = (lower .> 0) .| (upper .< 0)

    p = plot(xgrid, deriv;
        ribbon=(deriv .- lower, upper .- deriv),
        fillalpha=0.2, color=:steelblue, linewidth=2,
        label="f/ $(xvar)",
        xlabel=string(xvar), ylabel="derivative",
        title="d/d$(xvar)")
    hline!(p, [0.0]; linestyle=:dash, color=:grey50, label="")

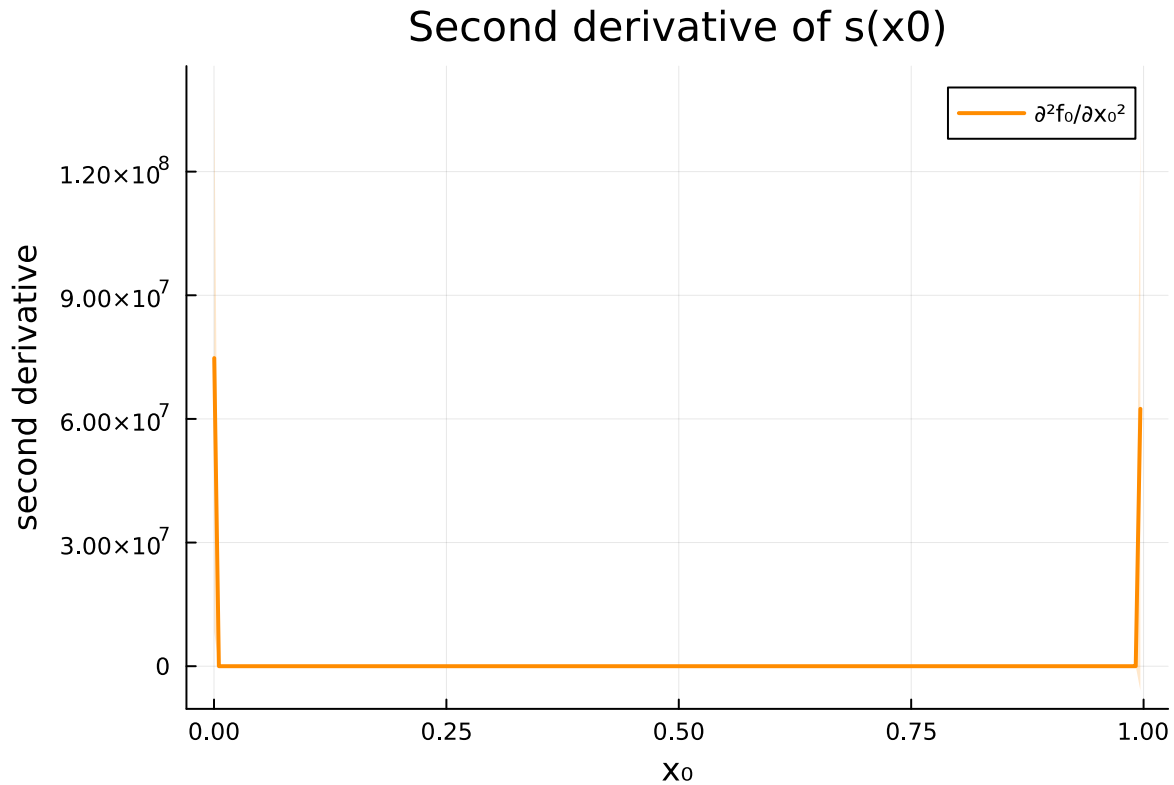
    # Mark significant regions with rug marks at bottom
    if any(sig)
        sig_x = xgrid[sig]
        scatter!(p, sig_x, zeros(length(sig_x));
            markersize=2, markerstrokewidth=0, color=:red,
            alpha=0.5, label="significant",
            markershape=:vline)
    end
    push!(plots_d, p)
end
plot(plots_d...; layout=(2, 2), size=(800, 700))

```



Second-order derivatives:

```
d2 = derivatives(m; order=2, select="x0", n=200)
plot(d2.x, d2.derivative;
     ribbon=(d2.derivative .- d2.lower, d2.upper .- d2.derivative),
     fillalpha=0.2, color=:darkorange, linewidth=2,
     label="  $\partial^2 f / \partial x^2$ ", xlabel="x", ylabel="second derivative",
     title="Second derivative of s(x0)", size=(600, 400))
```



Model diagnostics with appraise

`appraise` returns diagnostic data for the four standard residual plots: QQ plot, residuals vs linear predictor, histogram of residuals, and observed vs fitted values.

```
diag_data = appraise(m)
```

```
AppraiseData: 400 observations
Deviance residuals: min=-5.875, max=7.510
```

```
p1 = scatter(diag_data.qq_theoretical, diag_data.qq_sample;
             xlabel="Theoretical quantiles", ylabel="Standardized deviance residuals",
             title="QQ plot", label="", markersize=2, alpha=0.5, color=:steelblue)
lims = extrema(vcat(diag_data.qq_theoretical, diag_data.qq_sample))
plot!(p1, [lims[1], lims[2]], [lims[1], lims[2]];
       linestyle=:dash, color=:red, label="")

p2 = scatter(diag_data.linear_predictor, diag_data.residuals_deviance;
```

```

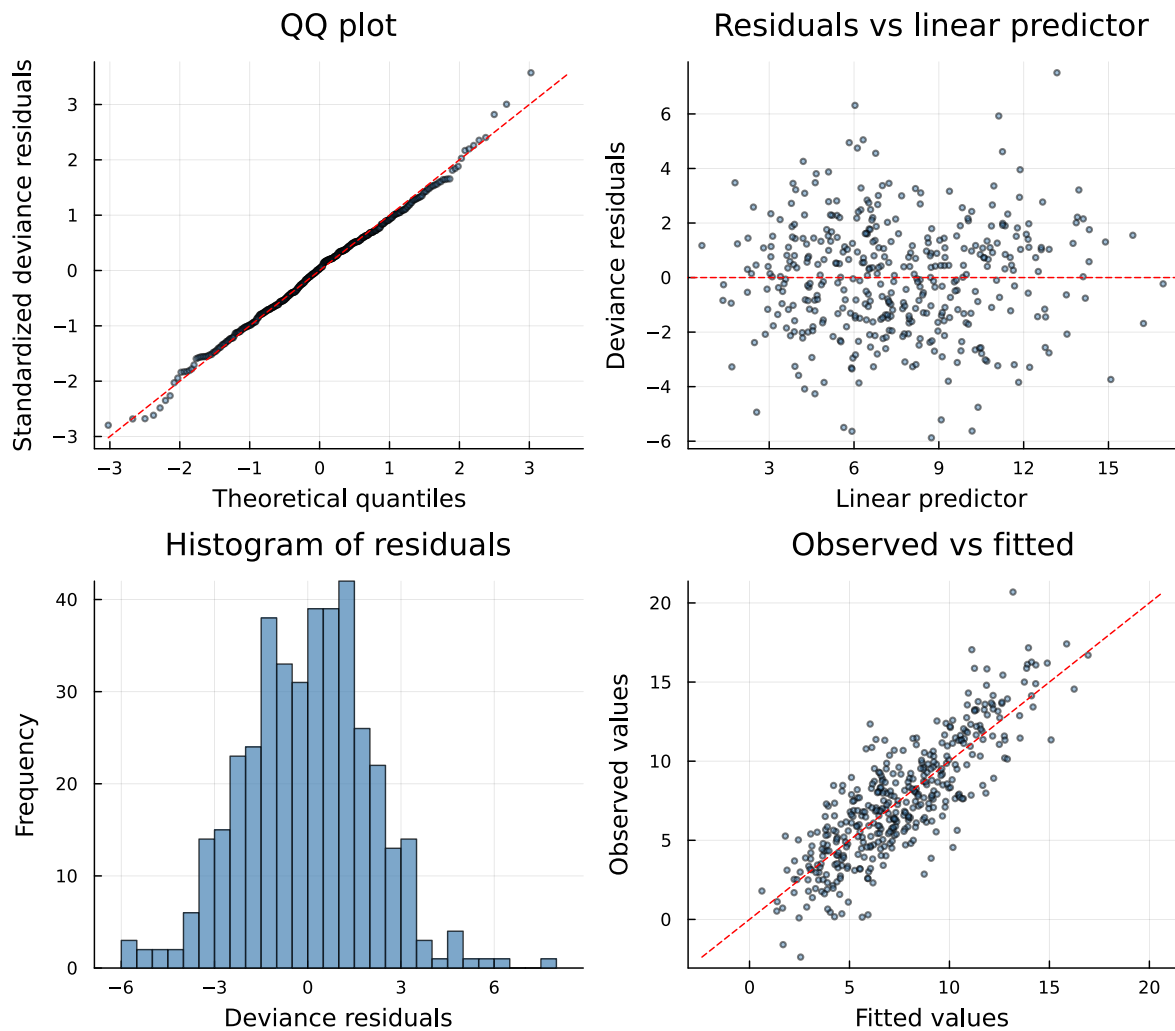
    xlabel="Linear predictor", ylabel="Deviance residuals",
    title="Residuals vs linear predictor", label="",
    markersize=2, alpha=0.5, color=:steelblue)
hline!(p2, [0.0]; linestyle=:dash, color=:red, label="")

p3 = histogram(diag_data.residuals_deviance;
    xlabel="Deviance residuals", ylabel="Frequency",
    title="Histogram of residuals", label="",
    bins=30, color=:steelblue, alpha=0.7)

p4 = scatter(diag_data.fitted, diag_data.observed;
    xlabel="Fitted values", ylabel="Observed values",
    title="Observed vs fitted", label="",
    markersize=2, alpha=0.5, color=:steelblue)
lims4 = extrema(vcat(diag_data.fitted, diag_data.observed))
plot!(p4, [lims4[1], lims4[2]], [lims4[1], lims4[2]];
    linestyle=:dash, color=:red, label="")

plot(p1, p2, p3, p4; layout=(2, 2), size=(800, 700))

```



Posterior simulation

Smooth samples

`smooth_samples` draws posterior realizations of individual smooth functions on an evaluation grid—useful for visualizing uncertainty in smooth shapes.

```
ss = smooth_samples(m; n=50, n_grid=100, seed=42)
```

Dict{String, Tuple{Vector{Float64}, Matrix{Float64}}} with 4 entries:

```
"s(x3,bs=cr)" => ([0.00127253, 0.0113417, 0.0214109, 0.03148, 0.0415492, 0.05...
```

```
"s(x2,bs=cr)" => ([0.00359141, 0.0136499, 0.0237083, 0.0337668, 0.0438252, 0.0538836, 0.063942, 0.0739999, 0.0840578, 0.0941157, 0.1041736, 0.1142315, 0.1242894, 0.1343473, 0.1444052, 0.1544631, 0.164521, 0.1745789, 0.1846368, 0.1946947, 0.2047526, 0.2148105, 0.2248684, 0.2349263, 0.2449842, 0.2550421, 0.2651, 0.2751579, 0.2852158, 0.2952737, 0.3053316, 0.3153895, 0.3254474, 0.3355053, 0.3455632, 0.3556211, 0.365679, 0.3757369, 0.3857948, 0.3958527, 0.4059106, 0.4159685, 0.4260264, 0.4360843, 0.4461422, 0.4562, 0.4662579, 0.4763158, 0.4863737, 0.4964316, 0.5064895, 0.5165474, 0.5266053, 0.5366632, 0.5467211, 0.556779, 0.5668369, 0.5768948, 0.5869527, 0.5970106, 0.6070685, 0.6171264, 0.6271843, 0.6372422, 0.6473, 0.6573579, 0.6674158, 0.6774737, 0.6875316, 0.6975895, 0.7076474, 0.7177053, 0.7277632, 0.7378211, 0.747879, 0.7579369, 0.7679948, 0.7780527, 0.7881106, 0.7981685, 0.8082264, 0.8182843, 0.8283422, 0.8384, 0.8484579, 0.8585158, 0.8685737, 0.8786316, 0.8886895, 0.8987474, 0.9088053, 0.9188632, 0.9289211, 0.938979, 0.9490369, 0.9590948, 0.9691527, 0.9792106, 0.9892685, 0.9993264, 1.0093843, 1.0194422, 1.0295, 1.0395579, 1.0496158, 1.0596737, 1.0697316, 1.0797895, 1.0898474, 1.0999053, 1.1099632, 1.12, 1.13, 1.14, 1.15, 1.16, 1.17, 1.18, 1.19, 1.2, 1.21, 1.22, 1.23, 1.24, 1.25, 1.26, 1.27, 1.28, 1.29, 1.3, 1.31, 1.32, 1.33, 1.34, 1.35, 1.36, 1.37, 1.38, 1.39, 1.4, 1.41, 1.42, 1.43, 1.44, 1.45, 1.46, 1.47, 1.48, 1.49, 1.5, 1.51, 1.52, 1.53, 1.54, 1.55, 1.56, 1.57, 1.58, 1.59, 1.6, 1.61, 1.62, 1.63, 1.64, 1.65, 1.66, 1.67, 1.68, 1.69, 1.7, 1.71, 1.72, 1.73, 1.74, 1.75, 1.76, 1.77, 1.78, 1.79, 1.8, 1.81, 1.82, 1.83, 1.84, 1.85, 1.86, 1.87, 1.88, 1.89, 1.9, 1.91, 1.92, 1.93, 1.94, 1.95, 1.96, 1.97, 1.98, 1.99, 2.0, 2.01, 2.02, 2.03, 2.04, 2.05, 2.06, 2.07, 2.08, 2.09, 2.1, 2.11, 2.12, 2.13, 2.14, 2.15, 2.16, 2.17, 2.18, 2.19, 2.2, 2.21, 2.22, 2.23, 2.24, 2.25, 2.26, 2.27, 2.28, 2.29, 2.3, 2.31, 2.32, 2.33, 2.34, 2.35, 2.36, 2.37, 2.38, 2.39, 2.4, 2.41, 2.42, 2.43, 2.44, 2.45, 2.46, 2.47, 2.48, 2.49, 2.5, 2.51, 2.52, 2.53, 2.54, 2.55, 2.56, 2.57, 2.58, 2.59, 2.6, 2.61, 2.62, 2.63, 2.64, 2.65, 2.66, 2.67, 2.68, 2.69, 2.7, 2.71, 2.72, 2.73, 2.74, 2.75, 2.76, 2.77, 2.78, 2.79, 2.8, 2.81, 2.82, 2.83, 2.84, 2.85, 2.86, 2.87, 2.88, 2.89, 2.9, 2.91, 2.92, 2.93, 2.94, 2.95, 2.96, 2.97, 2.98, 2.99, 3.0, 3.01, 3.02, 3.03, 3.04, 3.05, 3.06, 3.07, 3.08, 3.09, 3.1, 3.11, 3.12, 3.13, 3.14, 3.15, 3.16, 3.17, 3.18, 3.19, 3.2, 3.21, 3.22, 3.23, 3.24, 3.25, 3.26, 3.27, 3.28, 3.29, 3.3, 3.31, 3.32, 3.33, 3.34, 3.35, 3.36, 3.37, 3.38, 3.39, 3.4, 3.41, 3.42, 3.43, 3.44, 3.45, 3.46, 3.47, 3.48, 3.49, 3.5, 3.51, 3.52, 3.53, 3.54, 3.55, 3.56, 3.57, 3.58, 3.59, 3.6, 3.61, 3.62, 3.63, 3.64, 3.65, 3.66, 3.67, 3.68, 3.69, 3.7, 3.71, 3.72, 3.73, 3.74, 3.75, 3.76, 3.77, 3.78, 3.79, 3.8, 3.81, 3.82, 3.83, 3.84, 3.85, 3.86, 3.87, 3.88, 3.89, 3.9, 3.91, 3.92, 3.93, 3.94, 3.95, 3.96, 3.97, 3.98, 3.99, 4.0, 4.01, 4.02, 4.03, 4.04, 4.05, 4.06, 4.07, 4.08, 4.09, 4.1, 4.11, 4.12, 4.13, 4.14, 4.15, 4.16, 4.17, 4.18, 4.19, 4.2, 4.21, 4.22, 4.23, 4.24, 4.25, 4.26, 4.27, 4.28, 4.29, 4.3, 4.31, 4.32, 4.33, 4.34, 4.35, 4.36, 4.37, 4.38, 4.39, 4.4, 4.41, 4.42, 4.43, 4.44, 4.45, 4.46, 4.47, 4.48, 4.49, 4.5, 4.51, 4.52, 4.53, 4.54, 4.55, 4.56, 4.57, 4.58, 4.59, 4.6, 4.61, 4.62, 4.63, 4.64, 4.65, 4.66, 4.67, 4.68, 4.69, 4.7, 4.71, 4.72, 4.73, 4.74, 4.75, 4.76, 4.77, 4.78, 4.79, 4.8, 4.81, 4.82, 4.83, 4.84, 4.85, 4.86, 4.87, 4.88, 4.89, 4.9, 4.91, 4.92, 4.93, 4.94, 4.95, 4.96, 4.97, 4.98, 4.99, 5.0, 5.01, 5.02, 5.03, 5.04, 5.05, 5.06, 5.07, 5.08, 5.09, 5.1, 5.11, 5.12, 5.13, 5.14, 5.15, 5.16, 5.17, 5.18, 5.19, 5.2, 5.21, 5.22, 5.23, 5.24, 5.25, 5.26, 5.27, 5.28, 5.29, 5.3, 5.31, 5.32, 5.33, 5.34, 5.35, 5.36, 5.37, 5.38, 5.39, 5.4, 5.41, 5.42, 5.43, 5.44, 5.45, 5.46, 5.47, 5.48, 5.49, 5.5, 5.51, 5.52, 5.53, 5.54, 5.55, 5.56, 5.57, 5.58, 5.59, 5.6, 5.61, 5.62, 5.63, 5.64, 5.65, 5.66, 5.67, 5.68, 5.69, 5.7, 5.71, 5.72, 5.73, 5.74, 5.75, 5.76, 5.77, 5.78, 5.79, 5.8, 5.81, 5.82, 5.83, 5.84, 5.85, 5.86, 5.87, 5.88, 5.89, 5.9, 5.91, 5.92, 5.93, 5.94, 5.95, 5.96, 5.97, 5.98, 5.99, 6
```

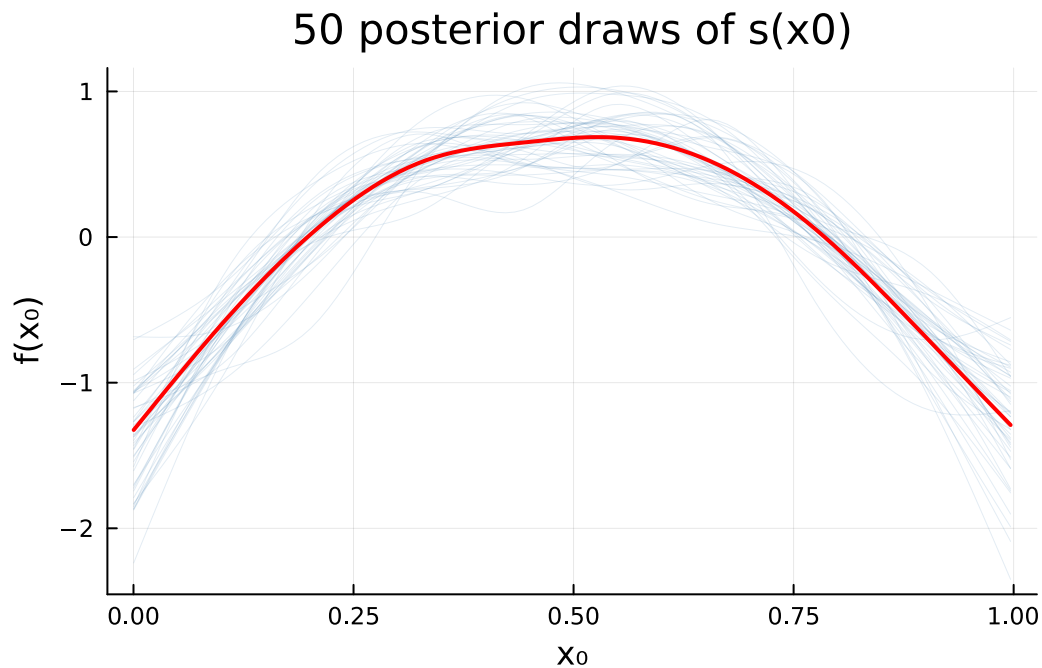
We show a “spaghetti plot” of posterior draws for $\mathbf{s}(\mathbf{x}_0)$:

```
label_x0 = m.smooths[1].spec.label
xgrid_ss, smat_ss = ss[label_x0]

p_ss = plot(xlabel="x ", ylabel="f(x)",
            title="50 posterior draws of s(x0)", legend=false)
for j in 1:size(smat_ss, 2)
    plot!(p_ss, xgrid_ss, smat_ss[:, j];
          color=:steelblue, alpha=0.15, linewidth=0.5)
end

# Overlay the point estimate
se_x0 = smooth_estimates(m; select="x0", n=100)
plot!(p_ss, se_x0.covariates[:x0], se_x0.estimate;
      color=:red, linewidth=2)

p_ss
```



Fitted samples

`fitted_samples` draws posterior samples of fitted values $\mathbf{X}\tilde{\beta}$, providing pointwise uncertainty in the mean response.

```
fs = fitted_samples(m; n=200, seed=42)
println("Fitted samples matrix size: ", size(fs))

lower95 = [quantile(fs[i, :], 0.025) for i in 1:size(fs, 1)]
upper95 = [quantile(fs[i, :], 0.975) for i in 1:size(fs, 1)]
println("Median 95% CI width: ", round(median(upper95 .- lower95), digits=3))
```

```
Fitted samples matrix size: (400, 200)
Median 95% CI width: 1.606
```

Concurvity

`concurvity` measures collinearity between smooth terms (the smooth analogue of multicollinearity). Values near 1 indicate high concurvity.

```
conc_full = concurvity(m; full=true)
```

```
4-element Vector{Float64}:
 0.12771464477604377
 0.13917946213220714
 0.1316305905556416
 0.16780562169356655
```

Pairwise concurvity matrix:

```
conc_pair = concurvity(m; full=false)
```

```
4×4 Matrix{Float64}:
 1.0      0.0660113  0.0855278  0.0507636
 0.0660113  1.0      0.062428  0.0807672
 0.0855278  0.062428  1.0      0.083151
 0.0507636  0.0807672  0.083151  1.0
```

Basis dimension check

`k_check` assesses whether the basis dimension k is adequate for each smooth. A high EDF-to- k' ratio suggests the basis may be too small.

```
kc = k_check(m)
for item in kc
    println("${item.label}: k' = ${item.k}, edf = ${round(item.edf, digits=2)}, ratio = ${round(item.ratio, digits=2)}")
end
```

```
s(x0,bs=cr): k' = 9, edf = 3.43, ratio = 0.381
s(x1,bs=cr): k' = 9, edf = 3.2, ratio = 0.355
s(x2,bs=cr): k' = 9, edf = 7.83, ratio = 0.87
s(x3,bs=cr): k' = 9, edf = 1.89, ratio = 0.21
```

Summary

GAM.jl provides a comprehensive set of gratia-style diagnostics:

Function	Purpose
<code>overview</code>	Tidy summary of smooth terms
<code>smooth_estimates</code>	Evaluate smooths on grids with SEs
<code>derivatives</code>	Finite-difference derivatives with CIs
<code>appraise</code>	Residual diagnostic data
<code>posterior_samples</code>	Posterior draws of coefficients
<code>fitted_samples</code>	Posterior draws of fitted values
<code>smooth_samples</code>	Posterior draws of smooth functions
<code>partial_residuals</code>	Partial residuals per smooth
<code>concurvity</code>	Smooth collinearity measures
<code>k_check</code>	Basis dimension adequacy